FSTT Local AI Assistant

Local AI with Ollama - lab report

1. Overview

This lab runs large language models entirely on a local machine with Ollama. It covers installing and querying models (qwen3.5, gemma3), calling the REST API from Python, structured JSON output, a mini-RAG pipeline (TF-IDF retrieval plus nomic-embed-text embeddings), image analysis with a multimodal model, a performance benchmark, and a final Flask web application with conversation history and response-time display.

```
Browser -> Flask/Python --+-- Conversation history --+
                          +-- Documents -> Retriever -+--> Ollama -> Local LLM
```

2. Deliverable layout

`source/` (app.py, chatbot.py, rag.py, benchmark.py, templates/), `documents/`, `screenshots/`, `README.md`, `report.pdf`.

3. Screenshots

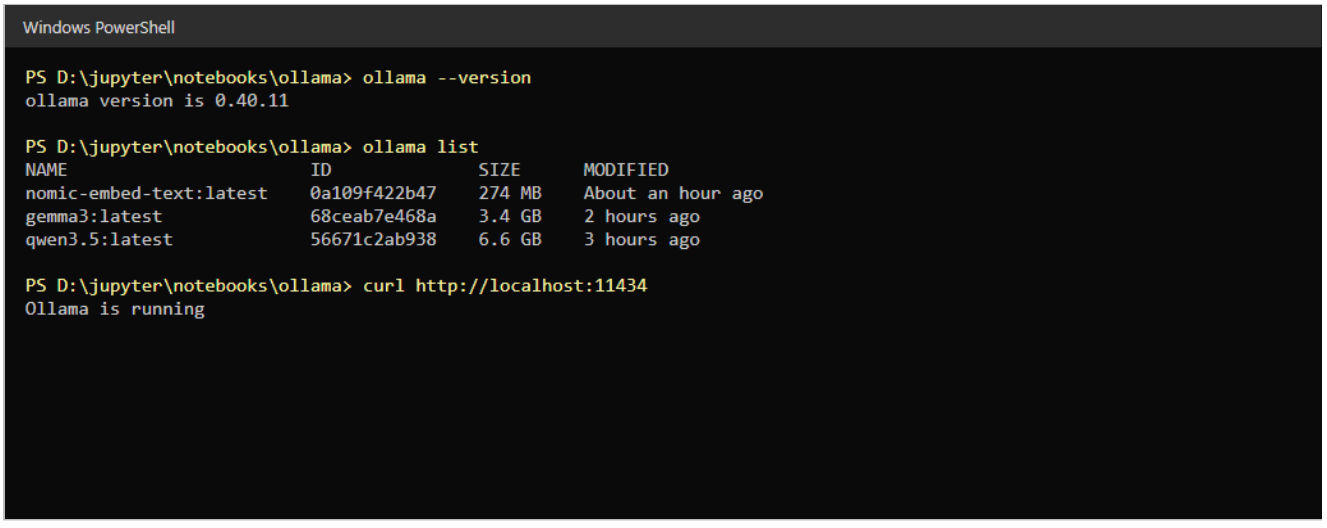


Figure 1 - Ollama installed and models available

```
Windows PowerShell

PS D:\jupyter\notebooks\ollama> ollama run gemma3 "Explain what a local LLM is in two sentences."
A local LLM (Large Language Model) is an AI model that runs directly on your computer, rather than relying on a remote
server. This means your data stays private and you have full control over the model's operation and performance.
```

Figure 2 - Running a model from the terminal

```
Windows PowerShell

PS ...\LocalAI_Lab\source> python chatbot.py
Local AI Assistant — type 'exit' to stop.

You: What is overfitting in machine learning?
AI: **Overfitting** occurs when a machine learning model learns the training data too well, capturing both underlying
patterns and random noise (irrelevant details).

### Key Characteristics
*   **Low Training Error:** The model performs exceptionally well on the dataset it was trained on.
*   **High Test Error:** It fails to generalize, performing poorly on new, unseen data.
*   **Complexity:** Usually results from a model that is too complex for the amount of data available (high variance).

### Common Solutions
*   **Simplify the Model:** Reduce features or decrease model depth/parameters.
*   **Regularization:** Add penalties (like L1/L2) to discourage overly complex weights.
*   **Early Stopping:** Halt training before performance on validation data starts degrading.
*   **Data Augmentation:** Increase dataset size by creating modified versions of existing samples.
*   **Dropout:** Randomly ignore neurons during neural network training (specific to deep learning).

You: Give me a one-line example of it.
AI: It is when a model memorizes the training data points perfectly (including noise and outliers) instead of learning
general patterns, leading to high accuracy on known data but poor performance on new, unseen examples.

You: exit
```

Figure 3 - Console chatbot with conversation memory

```
Python

Python (requests -> Ollama REST API)
import requests, time
t0 = time.perf_counter()
r = requests.post("http://localhost:11434/api/generate",
                  json={"model": "gemma3", "prompt": "In one sentence, what is Python used for in AI?", "stream":
False}).json()
print(r["response"].strip())
print("tokens:", r["eval_count"], "| tokens/s:", round(r["eval_count"]/(r["eval_duration"]/1e9),1), "| wall:",
round(time.perf_counter()-t0,2), "s")

>>> output
Python is a popular programming language in AI due to its extensive libraries and frameworks like TensorFlow and PyTorch,
which simplify the development of machine learning models and AI applications.
tokens: 34 | tokens/s: 153.2 | wall: 2.3 s
```

Figure 4 - Calling the Ollama REST API from Python

```
Python

RAG test: retriever + qwen3.5
Q: Which hardware is needed to train neural networks with many layers?
  retrieved: [('deep_learning.txt', 8.43)]
  A: GPUs are needed to train neural networks with many layers.

Q: What is supervised learning?
  retrieved: [('course.txt', 1.08), ('machine_learning.txt', 1.06)]
  A: Supervised learning uses labelled data to learn a mapping from inputs to outputs (classification, regression).

Q: Who won the 2018 World Cup?
  retrieved: []
  A: Information not available.
```

Figure 5 - Mini-RAG answers, including an out-of-document question

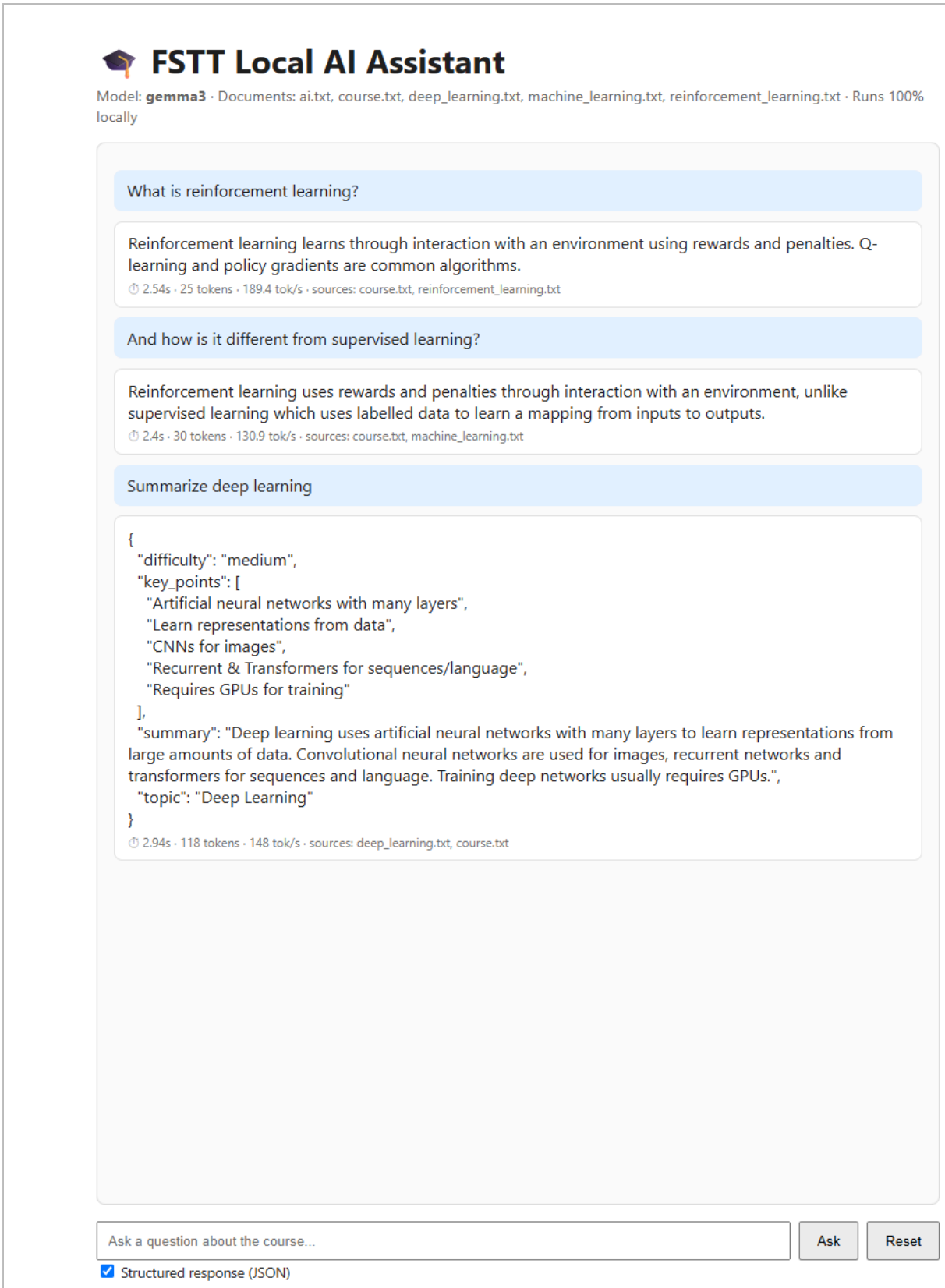


Figure 6 - FSTT Local AI Assistant web interface

4. Answers to the report questions

Q1. What is Local AI?

Local AI is just running the model on your own computer instead of sending requests to someone else's server. You download the weights once, and after that everything runs on your own CPU or GPU, so nothing leaves the machine and it still works with no internet.

Q2. Difference between a Cloud LLM and a Local LLM?

A cloud LLM like ChatGPT or Gemini runs on the provider's servers. You pay per token, your data travels over the internet, and you get very big models without owning any hardware. A local LLM runs on your own machine. It costs nothing per request and stays private, but you're limited to whatever fits in your RAM or VRAM (in my case models of 3 to 7 GB), so it's usually less capable, and you have to handle the setup and the speed yourself.

Q3. What role does Ollama play?

Ollama is the thing that makes running local models easy. It downloads and stores the models with `ollama pull`, loads them into memory (on the GPU if there is one), handles the quantized formats, and exposes a REST API on port 11434 (`/api/generate`, `/api/chat`, `/api/embed`). My notebook code, the RAG module and the Flask app all just talk to that API, so none of them have to deal with weights or inference directly.

Q4. What is model quantization?

Quantization means storing the model's weights with fewer bits, for example 4-bit integers instead of 16-bit floats. The file and the memory use get 2 to 4 times smaller and inference gets faster, and the price is a small loss of precision, so the quality drops slightly. It's the reason a multi-billion parameter model can run on a normal laptop or consumer GPU at all.

Q5. Why does model size affect memory consumption?

All the parameters have to sit in memory while the model runs, so memory is roughly the number of parameters times the bytes per weight, plus the KV cache, which grows with the context length. In my benchmark qwen3.5 used 5.61 GB loaded and gemma3 used 2.88 GB, about half, which matches their sizes on disk (6.55 GB vs 3.35 GB). If the model doesn't fit in VRAM, part of it ends up in system RAM and gets run by the CPU, and generation slows down a lot.

Q6. Why can a smaller model sometimes be preferable?

A smaller model loads faster (6.2 s vs 11.2 s for me), generates faster (209 vs 122 tokens/s), uses less memory and can run on weak hardware, even without a GPU. For simple jobs like short answers, summaries or RAG over text you supply, the context does most of the work, so the quality gap is small and the smaller model is the better deal. It also feels more responsive in a chat and uses less power.

Q7. Advantages of local inference?

Mainly privacy, since the data never leaves the machine. Beyond that: no per-token cost, it works offline, there's no network latency or rate limit, and I control the model version, so it can't change or get discontinued without me knowing. It's also a good fit for sensitive material like student records or company documents.

Q8. Disadvantages of local inference?

You're limited by your hardware. Only smaller or quantized models fit, so quality and context length fall short of the best cloud models. Speed depends on the GPU, big models can be very slow or not run at all, and the model takes RAM and power that other programs need. You also have to install, update and secure everything yourself, and it doesn't scale to many users out of the box.

Q9. Explain the architecture of your Python application.

The app is a Flask server (`app.py`) sitting in front of Ollama. The browser page (`templates/index.html`) sends the question to `/ask` with `fetch`. The server uses the retriever in `rag.py` (TF-IDF keyword search) to pick the top 2 documents from `documents/`, then builds the message list: a system prompt, the last 6 turns of the conversation, and the new question with its context. That goes to Ollama's `/api/chat`, with a JSON schema in `format` when structured output is ticked. The server sends back the answer, the sources, the response time and the tokens per second, and the page displays them. `/reset` clears the history. `chatbot.py` is a console version with memory, and `benchmark.py` measures the models.

Q10. Explain the role of the REST API.

The REST API is how my code talks to the model. Ollama is an HTTP server, so any program in any language can use the model by sending a JSON request (model, prompt or messages, options) and reading the JSON response, which has the text plus counters like `eval_count` and `eval_duration`. This keeps the client (notebook, Flask, console) separate from the model runtime, and I can switch models by changing one string. The browser and Flask use the same idea with `/ask` and `/reset`.

Q11. What is RAG?

RAG stands for Retrieval-Augmented Generation. Instead of trusting only what the model memorised during training, the system first looks up the documents most relevant to the question (TF-IDF keywords in my case, or embeddings with nomic-embed-text), puts them in the prompt, and tells the model to answer only from that. This keeps answers tied to my own course material, cuts down on hallucination, and lets me use private or recent information without retraining. The weak point is retrieval: in my test the follow-up question "how is it different from supervised learning?" got "Information not available." because the retriever only saw the words of the follow-up.

Q12. Difference between an LLM and a multimodal LLM?

A normal LLM takes text in and gives text out. A multimodal LLM also has encoders for other inputs, like a vision encoder for images and, in newer models, audio or video, which turn them into embeddings the language model can read alongside the text. In my Part 14 test, gemma3 got the image as base64 in the `images` field and described the page, including the text on it, which a text-only model can't do.

Q13. Compare two local models experimentally.

I ran the same prompt ("Write a 500-word explanation of reinforcement learning.") on both models, with a cold start each time:

Model	Disk (GB)	Loaded (GB)	Load (s)	Response (s)	Tokens	Tokens/s	Quality /5
qwen3.5	6.55	5.61	11.17	19.40	735	121.79	4
gemma3	3.35	2.88	6.24	12.48	854	209.25	3

Gemma 3 wins on every cost measure: about 1.7x faster, it answered in roughly 7 seconds less, and it used half the memory. Qwen 3.5 gave better structured answers and followed instructions more closely, which is why I scored it 4 against 3. Both were correct on the substance, so the extra cost buys structure and instruction-following, not correctness. It's only one run per model, so treat the numbers as a rough guide.

Q14. Which model would you deploy on a computer with limited RAM? Explain.

I'd deploy gemma3. It needs about 2.9 GB loaded against 5.6 GB for qwen3.5, which leaves room for the OS and the app on an 8 GB machine and avoids swapping or spilling out of VRAM. It also loads in about 6 s and answered in 12.5 s, and for RAG over supplied documents the quality was good enough. On an even smaller machine I'd drop to a 1B model. I'd only pick qwen3.5 if I had the memory to spare and strict formatting mattered.

Q15. What security risks exist when exposing a local LLM server to a network?

Ollama has no authentication by default, and my first Flask app even ran with `host='0.0.0.0'` and `debug=True`. If I set `OLLAMA_HOST=0.0.0.0` or otherwise expose the port, anyone on the network can use my GPU for free (or flood it with huge prompts), list, pull or delete my models through the API, and read whatever my app exposes, like documents and conversations. With RAG there's also prompt injection: an instruction hidden in a document can override the system prompt and leak other context. Flask's debug mode makes it worse, because the Werkzeug debugger allows remote code execution. To fix this I'd bind to 127.0.0.1 only (as the final app does), firewall the port, put a reverse proxy like nginx or Caddy with TLS, authentication and rate limiting in front, or use a VPN, turn debug off, and treat documents as untrusted input.